

KuraVet - Documentação Arquitetural de DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Integrantes:

- Pedro Henrique Luiz Alves Duarte – RM563405
- Guilherme Macedo Martins – RM562396
- Henrique Martins Oliveira - RM563620

Links Oficiais

- Vídeo Pitch da Solução: [Pitch](#)
- Repositório GitHub: [Repositorio](#)

1. Definição do Problema e Proposta de Valor

O mercado veterinário brasileiro atua predominantemente de forma episódica, onde o tutor aciona a clínica apenas em emergências ou para vacinas obrigatórias. Isso resulta em uma jornada de saúde fragmentada, baixa recorrência, enfraquecimento do vínculo entre clínica e tutor, e um baixo Lifetime Value (LTV) por animal. O componente de IA do KuraVet transforma dados isolados em uma jornada de cuidado contínuo e preventivo.

A inteligência artificial agrega valor aos seguintes atores:

- **Para o Tutor:** Elimina a carga mental do cuidado diário. A IA cruza dados de espécie, raça, idade e histórico para enviar recomendações ativas sobre nutrição, protocolos preventivos e continuidade medicamentosa.
- **Para a Clínica:** Aumenta o LTV e otimiza o tempo da equipe. A IA atua na priorização de atendimento, classificando a urgência de sintomas relatados e organizando a fila de teleconsulta de forma proativa para que a clínica atue antes do agravamento.
- **Para o Pet:** Garante bem-estar contínuo através do monitoramento preditivo adequado para cada fase biológica de sua vida.

2. Justificativa Tecnológica

Para suportar o ecossistema KuraVet, escolhemos uma abordagem híbrida que combina um Motor de Regras Inteligentes com uma LLM (Large Language Model) para NLP (Processamento de Linguagem Natural).

- **Prevenção de Alucinações (Motor de Regras):** No setor de saúde, a precisão clínica é inegociável e não tolera alucinações de modelos generativos puros. O motor de regras garante que o mapeamento clínico seja determinístico e seguro. Se o banco relacional indica um cão de 10 anos cardiopata, o sistema aciona protocolos terapêuticos rigidamente validados.
- **Interface e Triage (LLM/NLP):** A LLM atua estritamente na camada de comunicação. Ela recebe o relato informal de sintomas via chat no aplicativo, interpreta a intenção através de NLP e traduz essas informações em parâmetros estruturados. Isso permite ao sistema classificar a urgência sem depender que o tutor preencha formulários complexos.

3. Mapeamento de Dados (IoB/IoT)

A precisão da arquitetura depende de um fluxo constante de dados estruturados e comportamentais.

Dado	Origem	Estrutura	Utilização pela IA
Perfil do Pet (Espécie, raça, idade, peso)	Inserção do tutor no cadastro inicial via App Mobile.	Dados estruturados (JSON/Tabelas).	Definição de limites biológicos primários para o Motor de Regras disparar protocolos de vacinação e vermifugação.
Histórico Clínico e Vacinal	Inserção do veterinário via Painel Web e retido no Banco Oracle.	Dados estruturados (Relacional).	Fornece o contexto de saúde exato (como doenças crônicas ou cirurgias prévias) para garantir precisão nas recomendações.
Comportamento e Relatos (IoB)	Relatos de sintomas contínuos e hábitos diários	Dados não-estruturados (Texto via Chat).	A LLM processa a linguagem natural para extrair urgência clínica e

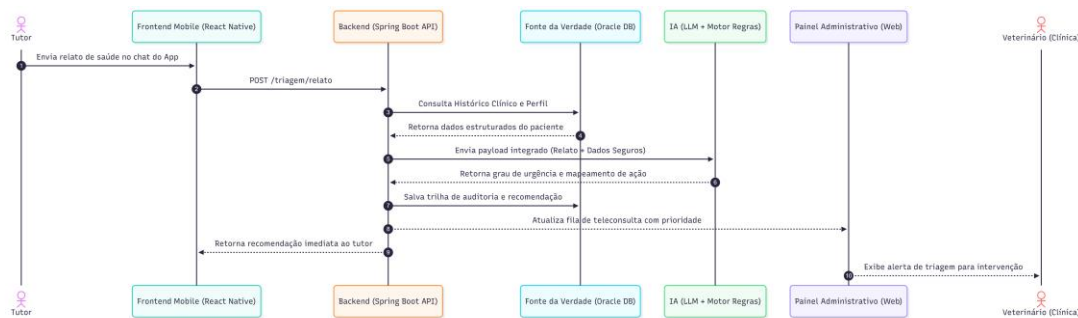
	inseridos pelo tutor no App.		retroalimentar a adesão terapêutica contínua.
--	------------------------------	--	---

4. Arquitetura e Fluxo de Dados

O ecossistema foi desenhado para garantir o trânsito seguro de informações entre as interfaces de usuário e a infraestrutura na nuvem.

1. O tutor insere um relato de sintoma na vitrine da aplicação (Mobile React Native).
2. A requisição é direcionada ao backend principal, nossa API estruturada em Java (Spring Boot).
3. A API consulta a fonte da verdade no banco Oracle para resgatar o histórico do paciente.
4. A API constrói um pacote contextualizado (Relato + Histórico) e o envia ao componente híbrido de IA.
5. O modelo de NLP processa o sintoma e o Motor de Regras avalia o risco, devolvendo a classificação de urgência e a diretriz clínica.
6. A API registra a trilha no banco de dados e simultaneamente notifica o tutor no aplicativo e atualiza a fila de triagem no painel administrativo web.

Diagrama:



5. Instruções de Uso

- Para implantar o ambiente e visualizar o fluxo de integração dos serviços:
7. Clone o repositório localmente utilizando git clone <https://github.com/KuraVet-Challenge-2026/iot-generative-ia.git>
 8. Certifique-se de possuir Docker e a Azure CLI configurados em seu ambiente.

9. Na raiz do projeto, execute `docker-compose up --build -d` para provisionar a API Spring Boot e o contêiner do banco de dados Oracle isolado de privilégios de root.
10. O tráfego da API REST estará liberado em `http://localhost:8080` e as views do painel da clínica em `http://localhost:8080/admin`.
11. Para rodar o projeto mobile, navegue até a pasta do client, execute `npm install` e inicie a vitrine do tutor através do `npx expo start`.

6. Tecnologias Utilizadas

- **App Mobile (Vitrine):** React Native executado via Expo com integração de autenticação Firebase.
- **Backend e Painel Web (Coração):** Java com Spring Boot servindo endpoints REST (JSON) e interfaces administrativas renderizadas via Thymeleaf.
- **Banco de Dados (Fonte da Verdade):** Oracle DB com lógica robusta em PL/SQL (Procedures/Functions) e Triggers para auditoria rigorosa.
- **Cloud & DevOps (A Esteira):** Implantação 100% containerizada com Docker, orquestrada na Azure (ACR e ACI).
- **Qualidade e Observabilidade:** Microserviços paralelos em .NET focados em testes (xUnit com padrão AAA) e coleta de logs com Serilog.
- **Inteligência Artificial:** Processamento hibridizado (Motor de Regras determinístico para segurança clínica + LLM para processamento semântico do tutor).

7. Resultados Parciais

Durante o ciclo de desenvolvimento focado na arquitetura, consolidamos os pipelines de CI/CD para os contêineres Docker alocados na Azure. A comunicação inicial entre a aplicação React Native, o monolito Spring Boot e as rotinas de banco de dados no Oracle DB já está estabelecida de forma estável. Os microsserviços satélites em .NET já integram verificações de integridade (health checks), garantindo fundações seguras para plugar a camada definitiva de inferência do modelo de Inteligência Artificial sem comprometer a confiabilidade do histórico médico.